Tổng quan về lỗ hỏng Broken Access Control

Access control là gì ?

Access Control (Kiểm soát truy cập) là cơ chế bảo mật quan trọng giúp xác định và quản lý quyền hạn của người dùng hoặc hệ thống trong việc truy cập và thao tác với tài nguyên. Đây là một trong những trụ cột cơ bản của bảo mật thông tin, đảm bảo rằng chỉ những đối tượng được ủy quyền mới có thể thực hiện các hành động cụ thể trên các tài nguyên nhất định.

Để hiểu rõ hơn về Access Control, chúng ta cần phân biệt hai khái niệm quan trọng là Authentication (Xác thực) và Authorization (Phân quyền).

Authen và Author là gì ? Phân biệt Authen và Author

Authentication (Xác thực)

Định nghĩa: Authentication là quá trình xác minh danh tính của người dùng hoặc hệ thống. Nó trả lời câu hỏi *"Bạn là ai?"* . Authentication Đảm bảo rằng người dùng thực sự là người mà họ tuyên bố là mình.

Phương thức phổ biến:

- Username và Password
- Sinh trắc học (vân tay, khuôn mặt)
- OTP (One-Time Password)
- Token, Certificate
- Multi-Factor Authentication (MFA)

Ví dụ: Khi bạn đăng nhập vào Facebook bằng email và mật khẩu, hệ thống đang thực hiện Authentication để xác nhận bạn là chủ sở hữu của tài khoản đó.

Authorization (Phân quyền)

Định nghĩa: Authorization là quá trình xác định quyền hạn và những gì người dùng được phép làm sau khi đã được xác thực. Nó trả lời câu hỏi *"Bạn được phép làm gì?"* . Authorization Kiểm soát và giới hạn các hành động mà người dùng có thể thực hiện trên hệ thống.

Các quyền phổ biến:

- Read (Đọc)
- Write (Ghi/Chỉnh sửa)
- Execute (Thực thi)
- Delete (Xóa)
- Admin (Quản trị)

Ví dụ: Sau khi đăng nhập Facebook (Authentication), bạn chỉ có thể chỉnh sửa bài viết của chính mình, nhưng không thể sửa bài viết của người khác. Đó là Authorization đang hoạt động để giới hạn quyền hạn của bạn.

So sánh Authentication và Authorization

Tiêu chí	Authentication (Xác thực)	Authorization (Phân quyền)
Câu hỏi	"Bạn là ai?"	"Bạn được phép làm gì?"
Thứ tự	Xảy ra trước	Xảy ra sau Authentication
Mục đích	Xác minh danh tính	Kiểm soát quyền truy cập
Phương thức	Password, OTP, Biometric	ACL, RBAC, ABAC
Có thể thay đổi	Thông tin xác thực ít thay đổi	Quyền hạn có thể thay đổi linh hoạt

Cách Access Control hoạt động

Access Control (kiểm soát truy cập) hoạt động dựa trên việc **xác định người dùng là ai**, **họ được phép làm gì**, và **được phép truy cập tài nguyên nào**. Dưới đây là ba cơ chế quan trọng giúp hệ thống quyết định quyền truy cập.

1. Kiểm soát theo chiều dọc

Kiểm soát truy cập theo chiều dọc là các cơ chế hạn chế quyền truy cập vào các chức năng nhạy cảm cho các loại người dùng cụ thể.

Với kiểm soát truy cập theo chiều dọc, các loại người dùng khác nhau có quyền truy cập vào các chức năng ứng dụng khác nhau.

Ví dụ : Trong 1 web viết Blog

Admin : Có quyền thêm , sửa , xóa Blog , tài khoản của bất cứ user nào dưới quyền

User : Chỉ có quyền Viết , Xóa , chỉnh sửa blog của chính bản thân , chỉ xem được thông tin cá nhân của chính mình

2. kiểm soát theo chiều ngang

Kiểm soát truy cập theo chiều ngang là các cơ chế hạn chế quyền truy cập vào tài nguyên cho những người dùng cụ thể.

Với kiểm soát truy cập theo chiều ngang, những người dùng khác nhau có quyền truy cập vào một tập con các tài nguyên cùng loại.

Ví dụ: Trong một ứng dụng Mua Sắm trực tuyến

- Tất cả người dùng đều có thể xem **giỏ hàng**, **đơn hàng**, **sản phẩm yêu thích**, **địa chỉ giao hàng**...
- Tuy nhiên:
  - **User A** chỉ xem được giỏ hàng và đơn hàng của **chính User A**.
  - **User B** chỉ xem được giỏ hàng và đơn hàng của **chính User B**.
  - User A **không thể truy cập** vào:
    - `/orders/12345` nếu đó là đơn hàng của User B
    - `/cart/b` nếu đó là giỏ hàng của người khác

→ Dù họ đều là **người dùng bình thường**, họ chỉ được truy cập **dữ liệu thuộc sở hữu của chính mình**.

3. Kiểm soát truy cập phụ thuộc ngữ cảnh

Kiểm soát truy cập phụ thuộc ngữ cảnh hạn chế quyền truy cập vào chức năng và tài nguyên dựa trên trạng thái của ứng dụng hoặc cách người dùng tương tác với ứng dụng.

Kiểm soát truy cập phụ thuộc ngữ cảnh ngăn người dùng thực hiện các hành động theo sai thứ tự.

Trong một **ứng dụng ngân hàng trực tuyến**, người dùng **không thể chuyển tiền** nếu:

- tài khoản của họ đang bị **đóng băng (frozen)**,
- hoặc họ chưa **xác thực OTP** trong phiên giao dịch,

- hoặc họ đã yêu cầu chuyển tiền nhưng phiên giao dịch **đã hết hạn**.

→ Các hành động bị hạn chế **không phải vì vai trò người dùng**, mà vì **trạng thái hiện tại** của phiên giao dịch hoặc hệ thống.

Lỗ hỏng Broken Access Control là gì ?

Broken Access Control là lỗ hổng bảo mật xảy ra khi hệ thống không thực thi đúng các chính sách kiểm soát truy cập, cho phép người dùng truy cập hoặc thao tác với tài nguyên mà họ không được phép. Theo OWASP Top 10 (2021), Broken Access Control đứng ở vị trí số 1 trong danh sách các lỗ hổng bảo mật nghiêm trọng nhất.

Hậu quả: Kẻ tấn công có thể truy cập dữ liệu nhạy cảm, thay đổi thông tin người dùng khác, chiếm quyền quản trị hệ thống, hoặc thực hiện các hành động trái phép. Điều này có thể dẫn đến rò rỉ thông tin, mất dữ liệu quan trọng và ảnh hưởng nghiêm trọng đến uy tín của tổ chức.

Ví dụ: Một người dùng bình thường có thể thay đổi URL từ `/user/profile?id=123` thành `/user/profile?id=456` để xem thông tin cá nhân của người dùng khác, hoặc truy cập vào trang quản trị `/admin` mà không có quyền hạn phù hợp.

Một số lỗ hỏng Broken Access Control

Lỗ hỏng BAC tồn tại khi người dùng có thể truy cập các tài nguyên hoặc thực hiện các hành động mà họ vốn không được phép.

1. Vertical Privilege Escalation (Leo thang đặc quyền theo chiều dọc)

Đây là lỗ hổng xảy ra khi **người dùng cấp thấp (low-privilege)** có thể truy cập vào **chức năng dành cho người dùng cấp cao (high-privilege)**. Tức là user thường có thể làm điều mà **chỉ admin mới được phép làm**.

Ví dụ: Một user bình thường truy cập được vào trang `/admin/dashboard` mà không cần có quyền admin hoặc thực hiện được các chức năng quản trị như xóa tài khoản người dùng khác ở trên hệ thống thì đó là lỗ hổng liên quan đến Leo thang đặc quyền theo chiều dọc.

2. Horizontal Privilege Escalation (Leo thang đặc quyền ngang)

Mô tả: Người dùng có thể truy cập hoặc thao tác với dữ liệu của người dùng khác ở cùng cấp độ quyền hạn.

Ví dụ: User A có thể xem, chỉnh sửa hoặc xóa thông tin cá nhân của User B bằng cách thay đổi tham số ID trong URL: `/user/profile?id=456`.

3. Insecure Direct Object Reference (IDOR)

Lỗi này xuất hiện khi mà hệ thống cho phép truy cập trực tiếp đến các đối tượng (file, database records) thông qua tham số có thể đoán được mà không kiểm tra quyền hạn. Là lỗ hỏng mặc dù thấy rất đơn giản nhưng hậu quả mà nó xảy ra là rất lớn.

Ví dụ: Thay đổi URL từ `/invoice/123.pdf` sang `/invoice/124.pdf` để tải xuống hóa đơn của người khác. Thay đổi `/ChuyenTien?uid1=1337&uid2=3609&value=36018` thành `/ChuyenTien?uid1=0&uid2=1337&value=10000000` nếu hệ thống không thử lí kĩ 2 tham số là uid1 và uid2 thì sẽ khiến cho bất cứ người dùng nào đều có quyền chuyển tiền từ bất kì tài khoản nào khác trên hệ thống về tải khoản mình mà không cần phải sở hữu tài khoản đó.

4. Forced Browsing (Truy cập cưỡng bức)

Forced Browsing xảy ra khi người dùng có thể tự ý truy cập vào các trang hoặc tài nguyên **không được liên kết trên giao diện**, nhưng vẫn tồn tại trên server và **không có bất kỳ kiểm soát truy cập nào**.

Tin tặc chỉ cần biết đúng URL là có thể vào được.

Ví dụ:

Một website có trang quản trị `/admin/` nhưng nút Admin bị ẩn. Tuy nhiên, nếu người dùng tự gõ đường dẫn `https://somewebsite.com/admin/`

mà hệ thống **không kiểm tra quyền**, user thường vẫn có thể truy cập được → dẫn đến mất kiểm soát truy cập.

5. Parameter-Based Access Control Bypass (Vượt mặt kiểm soát dựa trên tham số)

Một số chức năng bảo mật chỉ dựa trên tham số từ client như `"isAdmin=true"` hoặc `"role=user"`.

Nếu attacker có thể chỉnh sửa tham số này, họ có thể đổi vai trò của mình hoặc truy cập chức năng không được phép.

Ví dụ:

Khi gửi request cập nhật profile, client gửi:

```
POST /updateProfile
role=user&name=Hieu
```

Tin tặc sửa thành:

```
POST /updateProfile
role=admin&name=Hieu
```

Nếu backend **không xác thực role từ session** mà tin vào tham số client gửi lên → attacker có thể trở thành admin chỉ bằng cách sửa tham số.

Cách khắc phục

6. Missing Function Level Access Control (Thiếu kiểm soát truy cập ở mức chức năng)

Mô tả:

Đây là lỗ hổng xảy ra khi hệ thống chỉ **ẩn chức năng trên giao diện**, nhưng không kiểm tra quyền khi xử lý request.

Nó thường xảy ra trong các API như:

- `/deleteUser`
- `/updateRole`
- `/exportData`
- `/resetPassword`

Nếu API không kiểm tra quyền → người dùng thường chỉ cần gửi request là có thể thực hiện hành động của admin.

Ví dụ:

Trên giao diện, user không thấy nút "Xóa tài khoản người khác", nhưng họ vẫn có thể gửi request:

```
POST /deleteUser
id=55
```

và nếu server không kiểm tra quyền → user thường có thể xóa tài khoản bất kỳ.

Cách ngăn chặn, phòng chống

Để phòng tránh các lỗ hổng liên quan đến kiểm soát truy cập, hệ thống cần áp dụng phương pháp **phòng thủ nhiều lớp (defense-in-depth)** và tuân thủ các nguyên tắc quan trọng sau:

- Không được phép **dựa vào che giấu (obfuscation)** như một cơ chế kiểm soát truy cập.
- Trừ khi tài nguyên được thiết kế để công khai, hãy luôn **thiết lập chế độ từ chối truy cập theo mặc định (deny by default)**.
- Nên sử dụng **một cơ chế kiểm soát truy cập thống nhất và tập trung** cho toàn bộ ứng dụng để tránh sai lệch hoặc thiếu sót.
- Ở cấp độ code, lập trình viên cần **xác định rõ quyền truy cập cho từng tài nguyên**, đồng thời mặc định chặn truy cập khi quyền không được chỉ định.
- Thường xuyên **kiểm tra, rà soát và đánh giá toàn diện** các cơ chế kiểm soát truy cập để đảm bảo chúng hoạt động đúng như mong đợi và không bị bỏ sót.
- Tạo **Ma trận phân quyền** rõ ràng cho từng role/user để tránh lỗ hỏng này xảy ra